

SYSTEM DESIGN EXPERIMENT 2: HIGH-LEVEL DESIGN (HLD) OF NETFLIX

1. REQUIREMENTS (FUNCTIONAL & NON-FUNCTIONAL)

A. Functional Requirements (What the system must DO):

- **Video Streaming:** Users should be able to play/stream videos smoothly without buffering.
- **Adaptive Bitrate (ABR):** Video quality automatically adjusts (360p, 480p, 720p, 1080p, 4K) based on user's internet speed.
- **Search & Discovery:** Users can search movies/shows by title, genre, language, or actors, and browse the homepage.
- **Continue Watching (Playback State):** System must remember watch history and exact timestamp so user can resume anytime on any device.
- **User Accounts & Profiles:** Sign up, login, subscription plans, and multiple viewer profiles per account.
- **Video Upload & Transcoding (Admin/Creator):** Admins/studios can upload new movies which are automatically converted into different formats/resolutions.

B. Non-Functional Requirements (System Quality & Performance):

- **High Availability (99.99%):** Streaming service must always be online. If recommendations fail, video playback must still work.
- **Low Latency (Fast Start):** Video should start playing within < 1 second of pressing 'Play' (minimal buffering).
- **Scalability:** System must easily handle traffic spikes when a popular movie/series releases (millions of concurrent users).
- **Reliability & Fault Tolerance:** No single point of failure (SPOF). If one server or database crashes, traffic routes to another.
- **Data Consistency:**
 - *Strong Consistency:* For user account, billing, and payment records.
 - *Eventual Consistency:* For watch history, ratings, and view counts.

2. BACK OF THE ENVELOPE (BOE) ESTIMATION

A. Traffic & Active Users Assumptions:

- **Total Registered Subscribers:** 250 Million
- **Daily Active Users (DAU):** ~50 Million users/day (20% of total)
- **Peak Concurrent Viewers:** ~10 Million streams at peak evening hours
- **Average Watch Time:** 2 hours per active user per day

B. Storage Estimation (Video Files & Metadata):

- **Total Movie & Series Catalog:** 10,000 titles
- **Raw Master File Size:** ~30 GB per title (high quality studio master)
- **Multiple Resolutions & Formats:**
 - 1080p (FHD): ~2.5 GB
 - 720p (HD): ~1.2 GB
 - 480p (SD): ~600 MB
 - 360p (Mobile): ~300 MB
- Audio tracks & subtitles in multiple languages: ~400 MB
- **Total Transcoded Size per Title:** ~5 GB
- **Total Video Storage Needed:**
 $10,000 \text{ titles} \times 5 \text{ GB} = 50,000 \text{ GB} = \mathbf{50 \text{ TB (Terabytes)}}$
- **With Backup & Replication (3 regions):** $50 \text{ TB} \times 3 = \mathbf{150 \text{ TB in S3}}$

• User Metadata Storage (Database):

- User profile, settings, preferences: ~2 KB
- Watch history & bookmarks: ~8 KB
- Total per user = ~10 KB
- Total DB Size: $250\text{M users} \times 10 \text{ KB} = \mathbf{2.5 \text{ TB}}$

C. Bandwidth & Network Egress Estimation:

- **Average Stream Bitrate:** 2.5 Mbps (blended average of mobile & HD streams)
- **Peak Concurrent Streams:** 10,000,000 active streams
- **Total Peak Bandwidth Egress:**
 $10,000,000 \text{ users} \times 2.5 \text{ Mbps} = 25,000,000 \text{ Mbps} = \mathbf{25 \text{ Tbps (Terabits/sec)}}$

• **Key Takeaway:** Standard cloud servers CANNOT handle 25 Tbps directly without crashing! Hence, **CDN (Content Delivery Network / Open Connect)** is required. CDN edge caches handle >95% of video bandwidth locally at ISP level!

D. Requests Per Second (QPS) Estimation:

- **Search & Browsing API Requests:**
 - 50M DAU $\times$ 10 clicks/searches per day = 500,000,000 requests/day
 - Average QPS = $500,000,000 / 86,400 \text{ seconds} = \mathbf{5,800 \text{ QPS}}$
 - Peak QPS (2x average) = $\mathbf{12,000 \text{ QPS}}$
- **Playback Heartbeat Beacons:**
 - 10M concurrent streams send a progress beacon every 10 seconds
 - Heartbeat QPS = $10,000,000 / 10\text{s} = \mathbf{1,000,000 \text{ QPS}}$
 - Ingested via lightweight **Kafka cluster** so web servers aren't overloaded.

3. SYSTEM COMPONENTS USED & THEIR ROLES

1. Client Devices

- Smart TV, Mobile, Web Browser
- Displays UI & Movie catalog
- Runs ABR logic (switches quality dynamically based on Wi-Fi speed)
- Decrypts protected video streams

2. DNS & Load Balancer

- **DNS (Route 53):** Directs user to nearest data center based on geo-location.
- **Load Balancer (Nginx/ALB):** Distributes incoming API traffic equally across backend servers to prevent overload.

3. API Gateway

- Single entry point for all mobile & TV client requests.
- Verifies user authentication (JWT).
- Rate limiting (stops spam/DDoS).
- Routes requests to appropriate microservice.

4. Backend Microservices

- **Auth Service:** Login, Signup, Passwords.
- **User Service:** Profiles, Subscriptions.
- **Catalog Service:** Movie metadata & Search.
- **Playback Service:** Generates video link & finds closest CDN server.

5. Transcoding Pipeline

- Chunks raw movie files into 2-5 sec pieces.
- Encodes chunks into H.264, H.265, AV1.
- Creates 360p, 480p, 720p, 1080p, 4K.
- Generates Manifest (.mpd / .m3u8) file so client can switch bitrates.

6. CDN (Open Connect)

- Dedicated edge cache servers placed physically inside local ISPs (Airtel, Jio, Comcast).
- Stores popular video chunks near user.
- Streams 95%+ of video data directly without touching the central cloud!

MySQL / Relational DB (RDBMS)

Used for **Billing, Payments & Subscriptions**. Guarantees strict ACID properties and data consistency (no double billing).

NoSQL DB (Cassandra / MongoDB)

Used for **User Watch History & Bookmarks**. Fast write throughput for billions of viewing timestamps partitioned by User ID.

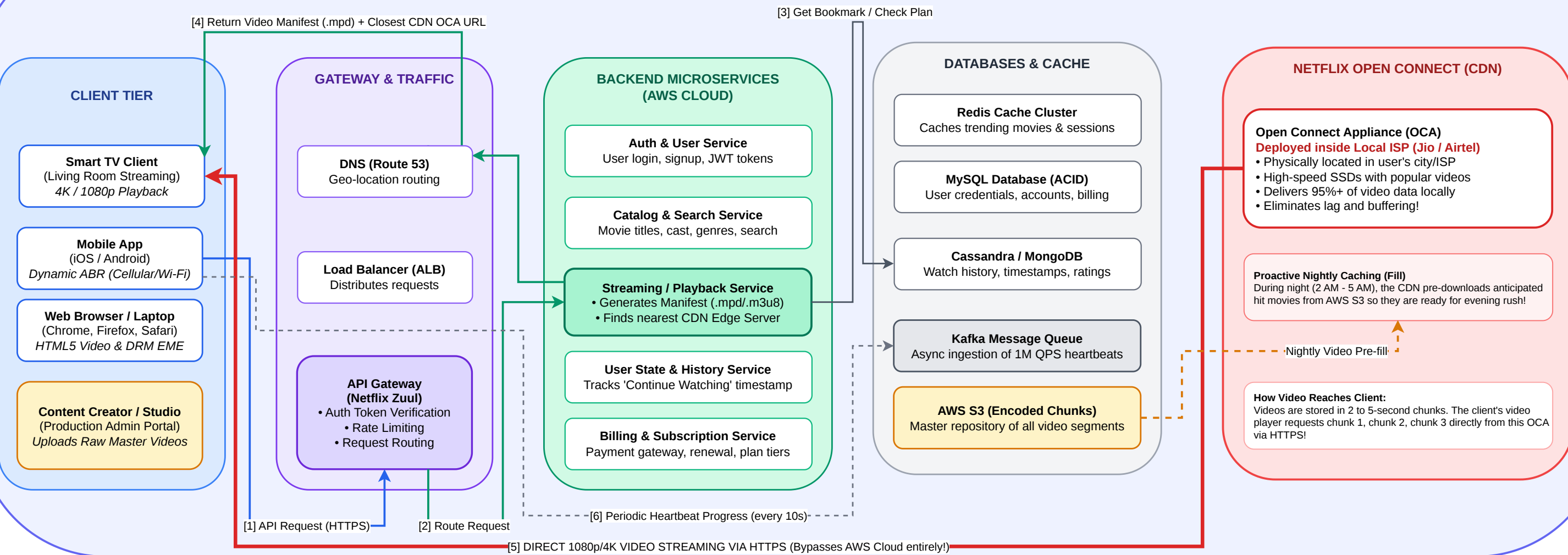
In-Memory Cache (Redis)

Caches **Trending Movies & User Profiles**. Provides sub-millisecond read response times and reduces database load by 80%.

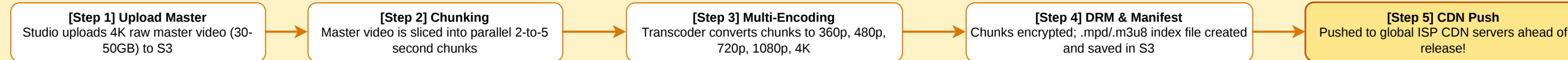
Object Storage (AWS S3)

Stores **Raw Master Videos & Encoded Chunks**. Highly durable, infinite cloud storage; serves as the master source for CDNs.

4. HIGH-LEVEL ARCHITECTURE DIAGRAM & END-TO-END SYSTEM FLOWS



OFFLINE VIDEO INGESTION & TRANSCODING PIPELINE (HOW A NEW MOVIE IS ADDED)



Why Transcoding is necessary: Different users have different screens (4-inch mobile vs 65-inch 4K TV) and different connection speeds (2G, 4G, Fiber Wi-Fi). Storing multiple encoded versions allows the client to switch quality on the fly!

STEP-BY-STEP USER WORKFLOWS (HOW THE SYSTEM CONNECTS TOGETHER)

FLOW 1: Browse & Search Flow

1. User opens Netflix app & searches or opens home feed.
2. Request passes through **DNS** and **Load Balancer** to **API Gateway**.
3. Gateway verifies JWT token and calls **Catalog Service**.
4. Catalog Service checks **Redis Cache**. On cache hit, returns in <5ms. If cache miss, reads from database and updates Redis.
5. User sees movie posters and list.

FLOW 2: Video Playback Initiation Flow

1. User clicks **PLAY** on a movie.
2. Client sends request to **Playback Service** via API Gateway.
3. Playback Service checks user subscription in **MySQL** and fetches resume timestamp from **Cassandra**.
4. Playback Service determines user's ISP & IP and selects the **closest Open Connect CDN server (OCA)**.
5. Returns a **Manifest (.mpd)** containing CDN URLs to the client.

FLOW 3: Real-Time Video Streaming & State Sync

1. Client bypasses cloud backend and connects **directly to ISP Open Connect CDN!**
2. Client requests 2-to-5 second video chunks via HTTPS.
3. If network slows down, client player automatically requests smaller 480p chunks (Adaptive Bitrate - ABR).
4. Every 10 seconds, client sends a lightweight heartbeat to **Kafka** to save viewing progress in **Cassandra** (Continue Watching).

